

Task 3B: Post-Refactoring Metrics Analysis

Relevant Metrics: Pre vs. Post Refactoring

Based on the nature of the changes, specifically decomposing large logic blocks and breaking architectural cycles. The following six metrics provide the most accurate representation of the design improvements :

1. **WMC**: Weighted Methods per Class
2. **CBO**: Coupling Between Objects
3. **LCOM**: Lack of Cohesion of Methods
4. **RFC**: Response for a Class
5. **CC**: Cyclomatic Complexity
6. **LOC**: Lines of Code

Reasoning for choosing the above metrics :

- **WMC, CC, and LOC** measure the reduction of logic and size in the former "Brain Class" (LuceneIndexManager) and "God Class" (WeblogEntry).
- **LCOM** captures the improvement in cohesion after splitting multi-purpose classes into specialized helpers.
- **CBO** tracks the elimination of cyclic dependencies in the User subsystem.
- **RFC** reflects the simplified communication paths after removing presentation and business logic from domain POJOs.

Subsystem	Metric	Initial (Before)	After Refactoring	Change Impact
Weblog and Content	WMC	134	123	-8.2% (Improved)
Weblog and Content	LOC	538	485	-9.8% (Improved)
User and Role	WMC	36	33	-8.3% (Improved)
User and Role	LOC	135	119	-11.8% (Improved)
Search Subsystem	WMC	47	22	-53.2% (Improved)
Search Subsystem	LOC	270	180	-33.3% (Improved)

Subsystem Discussion and Impact

Subsystem 1: Weblog and Content Subsystem (God Class & Layer Violation)

- **The Issue:** WeblogEntry.java acted as a God Class, containing both domain data and presentation logic (e.g., HTML formatting and URL generation).
- **Refactoring:** Extracted presentation logic from domain POJOs (URL generation and HTML formatting) into a dedicated ViewModel layer and centralized pagination logic into PaginationHelper.java.
- **Metric Change (WeblogEntry.java):**
 - **Initial (Before):** WMC: 134, LOC: 538.
 - **Post-Refactoring:** WMC: 123, LOC: 485.
- **Impact:** By removing methods like createAnchorBase (CC: 5) and displayContent (CC: 5) from the domain POJO, we achieved a "Pure Domain Model." The UI classes now depend on a centralized helper, which significantly reduces code duplication.

Subsystem 2: User and Role Management (Broken Hierarchy)

- **The Issue:** Broken Hierarchy across RollerPermission, GlobalPermission, ObjectPermission, WeblogPermission. Broken hierarchy / tight coupling. The base class knew about the child class. Subclasses like ObjectPermission and GlobalPermission had their own actions field while the parent conceptually owned actions.
- **Refactoring:** Different parts of the system were each keeping their own copy of permission information. The main permission class (RollerPermission) had a method that worked only with ObjectPermission. That means the base class depended on one of its child classes. The method was changed to work with RollerPermission itself instead of ObjectPermission.
- **Metric Change(WeblogPermission.java):**
 - **Initial (Before):** WMC: 10, LOC: 145, External Dependencies: 6.
 - **Post-Refactoring:** WMC: 12, LOC: 201, External Dependencies: 5.

Cyclic Dependent Modularization

- **Impact:** The impact of this change is that the permission system became more stable, flexible, and easier to maintain. By removing the direct dependency of the main permission class on a specific child class, the design now follows a clear top down structure where the base class is independent and reusable. This reduces the risk that changes in one permission type will accidentally break others, making the system safer to modify.
- **The Issue:** Cyclic dependencies were present because the User data class was directly calling business services and UI utilities for tasks like password encoding and permission checks. This broke the layered architecture rule that domain objects should not depend on

higher layers. Since User is widely used, these upward calls created several direct and transitive dependency cycles across managers, context classes, and caching components, increasing coupling and making testing and maintenance harder.

- **Refactoring:** Business logic was moved out of the User class into the UserManager in the business layer. Password reset and permission-check methods were relocated there. The User class was simplified into a pure data object with only fields and basic access methods. Imports of business and UI classes were removed from User, and UI components were updated to call UserManager instead. This followed a move-method and separation-of-concerns approach to restore proper layering.
- **Metric Change(User.java):**
 - **Initial (Before):** WMC: 36, LOC: 135, External Dependencies: 5.
 - **Post-Refactoring:** WMC: 33, LOC: 119, External Dependencies: 2.
- **Impact:** This refactoring removed some detected cyclic dependencies. The system became easier to maintain. Testability improved since User can now be tested without mocking services or UI utilities. Coupling decreased, cohesion increased, and the codebase became more stable and scalable.

Subsystem 3: Search Subsystem (Brain Class)

- **The Issue:** LuceneIndexManager.java was a "Brain Class" with a WMC of 47, handling everything from analyzer instantiation to result mapping.
- **Refactoring:** Decomposed the monolithic LuceneIndexManager into five cohesive collaborators (Lifecycle, Executor, Analyzer, Reader, and Mapper).
- **Metric Change (LuceneIndexManager.java):**
 - **Initial (Before):** WMC: 47, Max CC: 7, LOC: 270.
 - **Post-Refactoring:** WMC: 22 (Coordinator only), Max CC: 3.
- **Impact:** This subsystem saw the most dramatic shift in **LCOM**. By distributing logic across specialized classes, the main coordinator's responsibilities were reduced solely to delegation, making the search subsystem significantly easier to extend.

The Metric Trade-off: Why one improved while another worsened

Yes, this is an expected outcome of refactoring.

1. **Cohesion (LCOM) vs. Coupling (CBO):** When we break a **Brain Class** into five smaller classes, the **LCOM improves** (classes are more focused). However, the **CBO (Coupling) typically worsens** for the original class. The LuceneIndexManager now has more collaborators because it must explicitly delegate tasks to the five new helper classes.
2. **Complexity (CC) vs. Class Count:** Centralizing duplicated pagination logic into a PaginationHelper reduces the total **CC** of the UI layer but increases the **Number of**

Classes. We trade "vertical complexity" (massive, nested methods) for "horizontal complexity" (more files with simpler logic).

3. **Layering vs. LOC:** Fixing **Layer Violations** by introducing DTOs often increases the total **Lines of Code (LOC)** of the project due to the mapping logic required between layers. However, the **maintainability** improves because the code is no longer coiled.